Hochschule Offenburg

Winter Semester 2025/2026

PROJECT REPORT

End-to-End IoT Environmental Monitoring Pipeline:

Edge Processing, MQTT Telemetry, and Time-Series Visualization

Submitted by:

Hisham Salih Hassan Salih

TABLE OF CONTENTS

ABSTRACT vi

CHAPTER 1: INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Report Organization 2

CHAPTER 2: LITERATURE REVIEW & THEORETICAL BACKGROUND 3

2.1 IoT Architecture and Components 3

2.2 Environmental Sensors and Calibration 3

2.3 Communication Protocols for IoT 3

2.4 Time-Series Databases 3

2.5 Data Visualization Techniques 4

CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE 5

3.1 Overall System Architecture 5

3.2 Hardware Components 5

3.3 Software Stack 7

3.4 Data Flow Diagram 8

3.5 Network Architecture 9

CHAPTER 4: HARDWARE SETUP AND SENSOR CALIBRATION 10

4.1 Component Selection and Specifications 10

4.2 Circuit Design and Assembly 11

4.3 LDR Calibration Methodology 12

4.4 BME280 Sensor Configuration 14

4.5 Power Management 14

CHAPTER 5: EMBEDDED PROGRAMMING (ESP32 / ARDUINO) 16

5.1 Development Environment Setup 16

5.2 Firmware Architecture 16

5.3 Sensor Data Acquisition 16

5.4 WiFi and Network Management 17
5.5 MQTT Implementation 17
CHAPTER 6: MQTT PROTOCOL AND MESSAGE BROKER 18
6.1 MQTT Protocol Overview 18
6.2 Broker Configuration 18
6.3 Topic Structure and QoS Levels 18
6.4 Message Format Design 19
6.5 Testing with MQTT Explorer 19
CHAPTER 7: DATA PROCESSING WITH NODE-RED 20
7.1 Node-RED Architecture 20
7.2 Flow Design and Implementation 20
7.3 Data Validation and Transformation 21
7.4 Line Protocol Formatting 21
7.5 Error Handling and Logging 22
CHAPTER 8: TIME-SERIES DATABASE (INFLUXDB) 24
8.1 InfluxDB 2.x Architecture 24
8.2 Database Schema Design 24
8.3 Bucket Configuration and Retention Policies 25
8.4 Query Optimization 25
CHAPTER 9: DATA VISUALIZATION WITH GRAFANA 27
9.1 Dashboard Design Principles 27
9.2 Data Source Configuration 27
9.3 Panel Types and Configuration 27
9.4 Flux Query Development 30
9.5 Alert Configuration 31
CHAPTER 10: RESULTS AND ANALYSIS 32
10.1 System Performance Metrics 32
10.2 Data Quality Analysis 32
10.3 Environmental Insights 33

CHAPTER 11: DISCUSSION 34
11.1 Achievement of Objectives 34
11.2 Technical Challenges and Solutions 34
11.3 Lessons Learned 34
11.4 Limitations and Constraints 35
CHAPTER 12: CONCLUSIONS AND FUTURE WORK 36
12.1 Summary of Achievements 36
12.2 Future Work 36
REFERENCES 37
APPENDICES 38

LIST OF FIGURES

Figure 3.1: ESP32-WROOM-32E. 6
Figure 3.2: Light Dependent Resistor. 7
Figure 3.3: Data Flow Diagram. 9
Figure 7.1: Node-RED Data Processing Pipeline. 21
Figure 7.2: Error Handling and Logging 23
Figure 8.1: Raw Data Ingestion in InfluxDB Data Explorer. 25
Figure 9.1: Barometric Pressure Gauge with semantic threshold coloring. 28
Figure 9.2: Temperature trend analysis over 1 hour. 28
Figure 9.3: Instantaneous Light Intensity Bar Gauge. 29
Figure 9.4: Environmental Conditions Overlay 30

ABSTRACT

This report details the implementation of an end-to-end Internet of Things (IoT) pipeline for real-time environmental monitoring. An ESP32 microcontroller acts as the edge device, capturing light intensity via an analog LDR and temperature, pressure, and humidity via a digital BME280 sensor. To address analog nonlinearity, a custom log-log regression model is embedded in the firmware to calculate accurate Lux values. Telemetry is serialized into JSON payloads and transmitted via a decoupled MQTT architecture. On the backend, Node-RED parses the MQTT stream and formats it into Line Protocol for storage in InfluxDB, a specialized time-series database. Finally, Grafana provides dynamic data visualization and automated alerting, successfully demonstrating real-time incident response capabilities and establishing a robust foundation for scalable IoT deployments.

CHAPTER 1: INTRODUCTION

1.1 Background

The rapid growth of Internet of Things (IoT) technologies has enabled low-cost, real-time monitoring of environmental conditions across diverse application domains, including smart buildings, industrial automation, and environmental research. Modern architectures have shifted away from monolithic designs toward decoupled, distributed systems. Advances in edge computing (microcontrollers), asynchronous messaging protocols, and purpose-built time-series databases now allow sensor networks to collect, transmit, and analyze high-velocity data with minimal latency and human intervention.

1.2 Problem Statement

Traditional environmental monitoring systems are often rigid, expensive, and struggle to scale when handling high-frequency telemetry. Integrating heterogeneous hardware, specifically combining non-linear analog sensors with modern digital components, presents significant signal conditioning and calibration challenges at the edge. Furthermore, there is a critical need for a modular software pipeline capable of reliably ingesting, formatting, and visualizing this continuous time-series data in real time without bottlenecking or losing data integrity during network fluctuations.

1.3 Objectives

The primary objectives of this project are:

- **Hardware Integration:** To design an edge-processing node utilizing an ESP32 microcontroller to acquire data from both digital (BME280) and analog (LDR) sensors.
- **Sensor Calibration:** To implement empirical log-log regression modeling within the firmware to correct analog non-linearity.
- **Decoupled Telemetry:** To ensure reliable, non-blocking data transmission to the cloud utilizing the MQTT publish-subscribe protocol.

- **Data Engineering Pipeline:** To utilize Node-RED as a middleware translation layer to format raw JSON payloads into strict Line Protocol.
- **Time-Series Storage & Analysis:** To store the telemetry efficiently in InfluxDB and deploy a Grafana visualization layer for real-time monitoring and automated incident alerting.

1.4 Report Organization

This report is organized sequentially to mirror the data pipeline:

- **Chapter 2** presents the theoretical literature review.
- **Chapters 3 through 6** detail the system architecture, hardware calibration, embedded firmware, and MQTT network design.
- **Chapters 7 through 9** cover the backend data engineering, including Node-RED processing, InfluxDB storage, and Grafana visualization.
- **Chapters 10 through 12** present the results, technical discussion, and conclusions regarding future implementations.

CHAPTER 2: LITERATURE REVIEW & THEORETICAL BACKGROUND

2.1 IoT Architecture and Components

Modern IoT systems follow a structured pipeline that transforms physical phenomena into actionable digital insights. According to the foundational data engineering framework, this architecture consists of five distinct layers: Physical, Sensor/Analog, Digital, Network, and Storage/User [1]. A critical architectural consideration is the division of labor between Edge devices, which handle high-velocity tasks such as local filtering, and Cloud servers, which handle long-term storage.

2.2 Environmental Sensors and Calibration

The conversion of continuous physical signals into discrete digital data requires specific transduction methods. Analog sensors like Light Dependent Resistors utilize the photoresistive effect, governed by a power law ($R = A \cdot E^{-\gamma}$). Conversely, digital sensors like the BME280 utilize integrated piezo-resistive and band-gap elements. These sensors perform internal signal conditioning and ADC conversion, outputting calibrated digital data via the I2C protocol [2].

2.3 Communication Protocols for IoT

For the Network layer, the Message Queuing Telemetry Transport (MQTT) protocol is utilized due to its lightweight header footprint and asynchronous Publish/Subscribe model [3]. This architecture provides three critical decoupling dimensions: Space, Time, and Synchronization. Furthermore, MQTT guarantees message delivery through Quality of Service (QoS) levels ranging from 0 to 2.

2.4 Time-Series Databases

IoT sensor streams generate high-velocity, append-only records. Time-Series Databases (TSDBs) like InfluxDB are purpose-built for this task, utilizing columnar storage and heavy compression.

A core theoretical principle in TSDB design is managing Cardinality, storing low-cardinality metadata as indexed *Tags* and high-cardinality dynamic data as unindexed *Fields* [4].

2.5 Data Visualization Techniques

Data visualization transforms raw data into graphical representations that facilitate understanding and decision-making. Effective visualization techniques include line charts for trend analysis, bar charts for comparison, and dashboards combining multiple visual elements. Real-time visualization tools such as Grafana allow dynamic querying, alerting, and interactive exploration of time-series data [1].

CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

3.1 Overall System Architecture

The developed system follows a layered Internet of Things (IoT) architecture model consisting of perception, network, and application layers. This structured approach improves modularity, scalability, and maintainability. The architecture is divided into five functional layers:

1. **Sensing Layer (Physical/Analog):** Environmental sensors (LDR and BMe280) collect physical measurements.
2. **Edge Processing Layer (Digital):** The ESP32 microcontroller digitizes, filters, and preprocesses raw sensor data.
3. **Communication Layer (Network):** Data is transmitted via Wi-Fi using the MQTT protocol to a cloud-based broker.
4. **Data Management Layer (Storage):** Node-RED processes incoming messages and writes structured Line Protocol data into InfluxDB.
5. **Visualization Layer (User):** Grafana queries InfluxDB for real-time monitoring and historical analysis.

This separation of concerns ensures that each layer can be modified or extended independently without affecting the entire system.

3.2 Hardware Components

3.2.1 ESP32-WROOM-32E Microcontroller

The central processing unit of the system is the ESP32-WROOM-32E module. It was selected due to its integrated Wi-Fi capability, dual-core architecture, sufficient computational performance, and 12-bit analog-to-digital converter (ADC). The ESP32 performs sensor acquisition, local preprocessing, and MQTT message publishing [5].

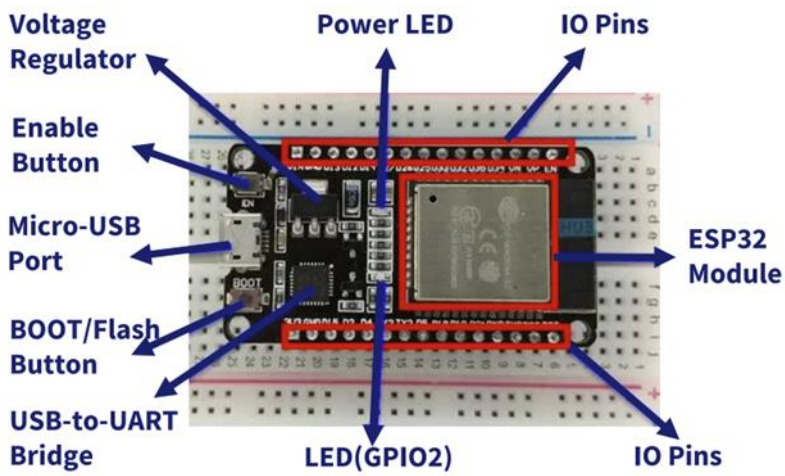

Figure 3.1: *ESP32-WROOM-32E.*

3.2.2 Light Dependent Resistor (NSL-19M51)

Ambient light intensity is measured using an NSL-19M51 light-dependent resistor (LDR). An LDR operates based on the principle of photoconductivity: its electrical resistance decreases as incident light intensity increases.

In darkness, the resistance is high, while under strong illumination, the resistance decreases significantly. The relationship between resistance (R) and illuminance (L) is non-linear and typically follows a power-law behavior:

$$R \propto L^{\alpha}$$

Where:

- **α**: is a sensor-specific constant.

Because the ESP32 measures voltage rather than resistance directly [5], the LDR is integrated into a voltage divider circuit. Changes in resistance produce corresponding voltage variations at the ADC input.

This non-linear behavior justifies the use of log-log linearization during calibration, as it allows transformation of the power-law relationship into a linear model suitable for regression analysis.

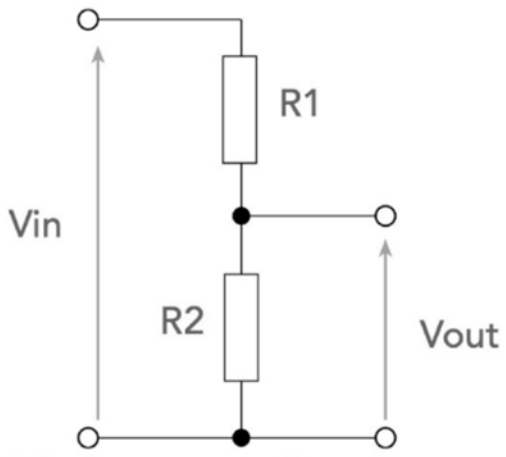

Figure 3.2: *Light Dependent Resistor.*

3.2.3 BME280 Environmental Sensor

The system also integrates a BME280 environmental sensor, which combines three sensing elements in a single device:

- Temperature sensor (bandgap-based measurement)
- Barometric pressure sensor (MEMS piezoresistive principle)
- Humidity sensor (capacitive measurement principle)

The BME280 communicates digitally via the I²C protocol, reducing susceptibility to analog noise. The sensor contains factory calibration coefficients stored in internal memory, allowing compensated and accurate digital output [2].

The integration of a digital multi-sensor reduces external signal conditioning requirements and simplifies firmware implementation.

3.3 Software Stack

The software architecture utilizes a modern, decoupled stack:

- **Embedded Firmware (C++):** Developed in the Arduino IDE, responsible for sensor sampling, JSON serialization, and network management.

- **MQTT Broker (Mosquitto):** Hosted externally (test.mosquitto.org), managing topic-based publish/subscribe routing.
- **Node-RED:** A middleware translation engine that subscribes to the MQTT broker and formats payloads for the database.
- **InfluxDB v2:** A specialized time-series database optimized for timestamped sensor data.
- **Grafana:** A visualization platform configured with Flux queries for dynamic dashboarding and active alerting.

This stack was selected due to its open-source availability, modular design, and suitability for time-series IoT applications.

3.4 Data Flow Diagram

The data flow of the system follows a sequential pipeline:

1. Sensors measure environmental parameters.
2. The ESP32 samples data (averaging the LDR and reading the BME280 via I2C).
3. Measurements are packed into a structured JSON payload with an ISO timestamp.
4. The ESP32 publishes the payload to the Mosquitto broker on topic **iot/sensors/group01/data**.
5. Node-RED subscribes to the topic, parses the JSON, and constructs InfluxDB Line Protocol.
6. Node-RED sends an HTTP POST request to the remote InfluxDB API.
7. Grafana queries InfluxDB to render dashboards and evaluate alert conditions.

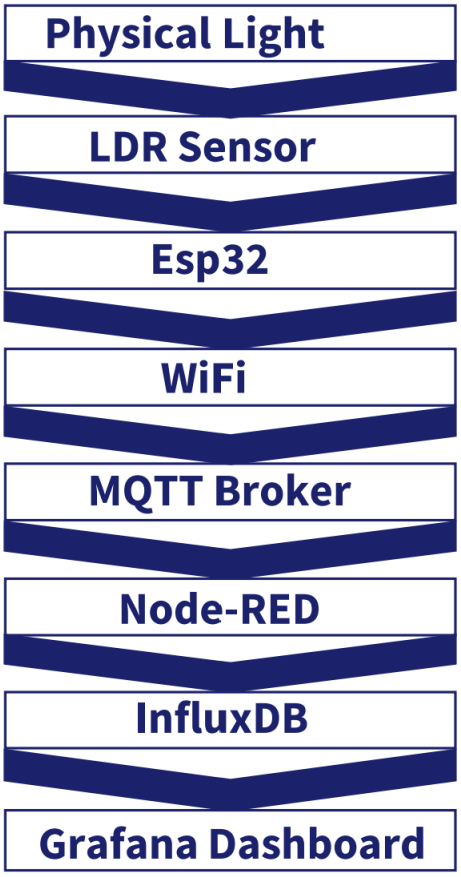

Figure 3.3: Data Flow Diagram.

3.5 Network Architecture

Unlike isolated local deployments, this system utilizes a Hybrid Cloud Architecture. The ESP32 operates in Wi-Fi station mode to connect to the internet.

- **Publisher:** The ESP32 connects outbound to a public cloud MQTT broker (**test.mosquitto.org**).
- **Processing & Storage:** Node-RED and InfluxDB are hosted on a remote academic server.

Advantages of this Architecture:

- **Global Accessibility:** Dashboards and data streams can be accessed from anywhere.
- **Decoupling:** If the storage server goes offline, the ESP32 continues publishing to the cloud broker independently.

CHAPTER 4: HARDWARE SETUP AND SENSOR CALIBRATION

4.1 Component Selection and Specifications

4.1.1 NSL-19M51 Light Dependent Resistor (LDR)

The NSL-19M51 LDR was selected for ambient light measurement due to its wide dynamic range, sensitivity in the visible spectrum, low cost, and simple analog interfacing capability. The sensor exhibits a nonlinear resistance–illuminance relationship, typically modeled as a power-law function.

Key characteristics:

- Passive photoconductive sensor
- Resistance decreases with increasing light intensity.
- Suitable for voltage divider implementation

Because the sensor outputs resistance rather than voltage, signal conditioning is required before analog-to-digital conversion.

4.1.2 BME280 Environmental Sensor

The BME280 was selected as a compact multi-parameter environmental sensor integrating:

- Temperature sensing (bandgap principle)
- Barometric pressure sensing (MEMS piezoresistive element)
- Humidity sensing (capacitive principle)

The sensor provides fully digital output via the I^2C protocol and includes factory calibration coefficients stored in internal memory. This reduces the need for external compensation circuitry and improves measurement stability.

4.1.3 ESP32-WROOM-32E

The ESP32-WROOM-32E microcontroller was selected due to:

- Integrated Wi-Fi capability
- 12-bit ADC resolution
- Sufficient processing capability for real-time data acquisition
- Low power operation at 3.3 V

The 12-bit ADC provides digital values from 0 to 4095, enabling fine voltage resolution for analog sensing.

4.2 Circuit Design and Assembly

4.2.1 LDR Voltage Divider Design

The LDR was integrated using a voltage divider configuration consisting of:

- Fixed resistor: **10 kΩ**
- Supply voltage: **3.3 V**
- Output connected to ESP32 ADC input

The output voltage is given by the voltage divider equation:

$$\boldsymbol{V_out = V_cc \cdot (R_LDR / (R_fixed + R_LDR))}$$

where:

- V_cc = **3.3 V**
- R_fixed = **10 kΩ**
- R_LDR = light-dependent resistance

As illumination increases, R_LDR decreases, producing a measurable voltage variation at the ADC input.

4.2.2 ADC Conversion

The ESP32 12-bit ADC converts the analog voltage to a digital value according to:

$$\mathbf{ADC = (V_out / V_ref) \cdot 4095}$$

Assuming V_ref = 3.3 V, the theoretical voltage resolution is:

$$\mathbf{\Delta V = 3.3\ V / 4096 \approx 0.8\ mV}$$

This defines the minimum detectable voltage change.

Suggested Figure Placement: Insert the LDR voltage divider schematic here.

4.3 LDR Calibration Methodology

The LDR exhibits a nonlinear resistance–illuminance relationship that follows a power-law model:

$$R_LDR = K \cdot L^{\alpha}$$

where:

- L = illuminance (lux)
- k and α = sensor-specific constants

To linearize the relationship, a logarithmic transformation was applied:

$$\mathbf{log(R_LDR) = log(k) - \alpha\ log(L)}$$

This transformation converts the nonlinear power-law relationship into a linear equation of the form:

$$\mathbf{y = a + bx}$$

where:

- y = log(R_LDR)

- $x = \log(L)$

Calibration was performed by collecting paired measurements of ADC values and reference illuminance values. Linear regression in log-log space was then applied to estimate the model parameters.

Calibration Curve Representation

The calibration results are best represented using a log-log plot of illuminance versus resistance (or derived resistance values from ADC readings).

When plotted on logarithmic axes:

- The x-axis represents log(L)
- The y-axis represents log(R_LDR)

The resulting data points form an approximately linear trend. The slope corresponds to $-\alpha$, and the intercept corresponds to log(k).

Suggested Figure Placement: Insert the calibration curve here showing:

- Experimental data points in log-log scale
- Linear regression fit line
- Regression equation
- Coefficient of determination (R^2 value)

Including the regression equation and R^2 value quantitatively demonstrates model accuracy and validates the power-law assumption.

This methodology:

- Reduces systematic nonlinearity error
- Improves estimation accuracy
- Enables conversion from ADC readings to lux values

4.4 BME280 Sensor Configuration

BME280 Sensor Configuration

The BME280 sensor was configured using the I²C communication protocol.

4.4.1 Operating Mode

The sensor was set to normal mode, enabling continuous periodic measurement without requiring manual triggering. This configuration supports real-time monitoring applications.

4.4.2 Internal Compensation

The BME280 includes factory calibration coefficients stored in non-volatile memory. Raw sensor measurements are internally compensated using these coefficients before digital output is transmitted to the ESP32.

The use of a digital sensor eliminates the need for external signal conditioning and significantly reduces susceptibility to electrical noise compared to analog sensors.

Suggested Figure Placement: Insert BME280 I²C connection diagram here.

4.5 Power Management

The system operates at a supply voltage of **3.3 V**, compatible with both the ESP32 and connected sensors.

Key considerations include:

- Stable 3.3 V regulation
- Minimization of voltage ripple
- Consistent ADC reference voltage

Since the ESP32 ADC reference is directly related to supply voltage, fluctuations in **V_cc** may introduce measurement error in the LDR readings. Therefore, maintaining stable power delivery is essential for accurate analog measurements.

The BME280 benefits from low power consumption and efficient normal-mode operation, making the overall system suitable for continuous environmental monitoring applications.

Comparison Between Analog and Digital Sensing Approaches

The implemented system contrasts two fundamental measurement methodologies.

- The analog LDR requires external signal conditioning, ADC conversion, and mathematical regression modeling. It remains susceptible to quantization error, resistor tolerance, and supply voltage variations.
- Conversely, the digital BME280 utilizes internal compensation algorithms and digital I2C communication, simplifying implementation and improving noise immunity, albeit with less transparency regarding the internal correction mathematics.

CHAPTER 5: EMBEDDED PROGRAMMING (ESP32 / ARDUINO)

5.1 Development Environment Setup

The firmware was developed using the Arduino IDE, utilizing the ESP32 board support package. To handle the complex hardware interfacing and data formatting, several third-party libraries were integrated:

- `Adafruit_BME280.h` and `Wire.h` for I2C sensor communication.
- `WiFi.h` and `PubSubClient.h` for network and MQTT management.
- `ArduinoJson.h` for dynamic memory allocation and payload serialization.

5.2 Firmware Architecture

The system architecture abandons the traditional, blocking `delay()` function in favor of a non-blocking state machine utilizing the hardware timer (`millis()`). This ensures the ESP32 can maintain its MQTT connection in the background without freezing during the 20-second sleep intervals between sensor readings.

```
// Non-blocking sampling logic
unsigned long currentTime = millis();
if (currentTime - lastSampleTime >= SAMPLE_INTERVAL) {
  lastSampleTime = currentTime;
  readSensors();
  publishSensorData();
}
```

5.3 Sensor Data Acquisition

Data acquisition is divided into digital and analog sub-routines. The BME280 is queried directly via I2C. For the analog LDR, the firmware implements software oversampling to mitigate hardware electrical noise. The system reads the ADC 20 times and calculates an average before applying the derived power-law calibration math to output an accurate Lux value:

```
// Power law conversion applied to averaged ADC
float lux = LDR_COEFF_A * pow(adc_value, LDR_COEFF_B) + LDR_COEFF_C;
```

5.4 WiFi and Network Management

The ESP32 operates in Station (STA) mode. Upon boot, it enters a while loop, attempting to connect to the local Wi-Fi gateway. Crucially, once connected, the system immediately initializes a Network Time Protocol (NTP) sync via `pool.ntp.org` (configured for GMT+1). This ensures that all sensor data is stamped with an accurate ISO 8601 timestamp at the edge, rather than relying on the cloud server to guess when the data was generated.

5.5 MQTT Implementation

The `PubSubClient` library manages the MQTT connection to the broker. To handle the payload, the firmware utilizes `ArduinoJson` to construct a hierarchical data structure. A `StaticJsonDocument` allocates memory securely without heap fragmentation. The system nests the sensor data and appends critical system diagnostics (such as Free Heap memory and Wi-Fi RSSI) before serializing it into a character buffer for transmission over the network.

```
// JSON construction for nested payload
JsonObject BME280 = sensors.createNestedObject("BME280");
BME280["temperature"] = round(currentData.temperature * 100) / 100.0;
```

CHAPTER 6: MQTT PROTOCOL AND MESSAGE BROKER

6.1 MQTT Protocol Overview

The system utilizes the Message Queuing Telemetry Transport (MQTT) protocol for data transmission. This lightweight, publish-subscribe messaging transport is ideal for resource-constrained IoT edge devices. By decoupling the ESP32 (Publisher) from the Node-RED backend (Subscriber), the architecture ensures that sensor sampling operations are not blocked by database write latencies.

6.2 Broker Configuration

The ESP32 acts as an MQTT client connecting to a public cloud broker (`test.mosquitto.org`) over the standard unencrypted TCP port `1883`. The firmware implements a robust, non-blocking reconnection routine. If the Wi-Fi or broker connection drops, the system waits 5,000 milliseconds before attempting a reconnection, preventing the microcontroller from entering an infinite blocking loop.

```
if (!mqtt_client.connected()) {
  unsigned long now = millis();
  if (now - lastReconnectAttempt > 5000) {
    lastReconnectAttempt = now;
    // Reconnect logic...
```

6.3 Topic Structure and QoS Levels

The project implements a hierarchical topic namespace, allowing for precise data filtering and future scalability. The structure follows the pattern `iot/sensors/group01/#`:

- **Data Topic:** `iot/sensors/group01/data` (High-frequency sensor telemetry).
- **Config Topic:** `iot/sensors/group01/config` (Subscribed by the ESP32 to receive remote updates).
- **Status Topic:** `iot/sensors/group01/status` (System health and IP address).

Retained Messages: The Status topic utilizes a "Retained Message" flag (`true`). This ensures that if a new subscriber (e.g., a dashboard) connects to the broker, it immediately receives the last known "online" status and IP address of the ESP32 without waiting for the next publishing cycle.

```
// Publishing status with the Retained flag enabled
mqtt_client.publish(topic_status, jsonBuffer, true);
```

6.4 Message Format Design

To ensure the backend system can reliably parse the telemetry, data is serialized into a structured JSON (JavaScript Object Notation) payload using the `ArduinoJson` library. A `StaticJsonDocument` is utilized to pre-allocate memory on the stack, preventing heap fragmentation over long uptimes.

The payload is strictly hierarchical, nesting specific sensor domains (`BME280, ldr`) and operational metrics (`system`) under a single parent object, accompanied by an ISO 8601 timestamp:

```
// Nested JSON structure construction
JsonObject BME280 = sensors.createNestedObject("BME280");
BME280["temperature"] = round(currentData.temperature * 100) / 100.0;
BME280["pressure"] = round(currentData.pressure * 100) / 100.0;
```

6.5 Testing with MQTT Explorer

Prior to integrating the Node-RED processing pipeline, the MQTT communication was validated using MQTT Explorer. This diagnostic tool was used to subscribe to the `iot/sensors/group01/#` wildcard topic. It verified that the JSON payloads were formatted correctly, the timestamps were accurately synchronized via NTP, and the retained status messages persisted on the broker as designed.

CHAPTER 7: DATA PROCESSING WITH NODE-RED

7.1 Node-RED Architecture

The system utilizes Node-RED as a middleware translation layer between the raw MQTT data stream and the InfluxDB storage engine. The flow is designed to be lightweight and stateless, ensuring high throughput for the 20-second sensor reporting interval. The architecture follows a linear "Input-Process-Output" model:

1. **Input:** MQTT Subscriber node listening to **iot/sensors/group01/data**.
2. **Processing:** A JavaScript function node ("Simple Format") for parsing and type conversion.
3. **Output:** An HTTP iningrafagRequest node sending data directly to the InfluxDB v2 API.

7.2 Flow Design and Implementation

The flow was implemented to handle the nested JSON structure generated by the ESP32. Upon receiving a message, the **MQTT In** node (QoS 1) passes the payload to the processing function. Unlike standard implementations that rely on "InfluxDB Batch" nodes, this project utilizes a custom JavaScript implementation to maximize control over the data structure. The logic extracts specific sensor branches (`msg.payload.sensors.BME280` and `msg.payload.sensors.ldr`) and flattens them into a unified variables list.

As illustrated in **Figure 7.1**, the flow logic is linear, the flow subscribes to the MQTT topic **'iot/sensors/group01/data'**, processes the JSON payload using a custom JavaScript function, and transmits the formatted Line Protocol to InfluxDB via HTTP POST.

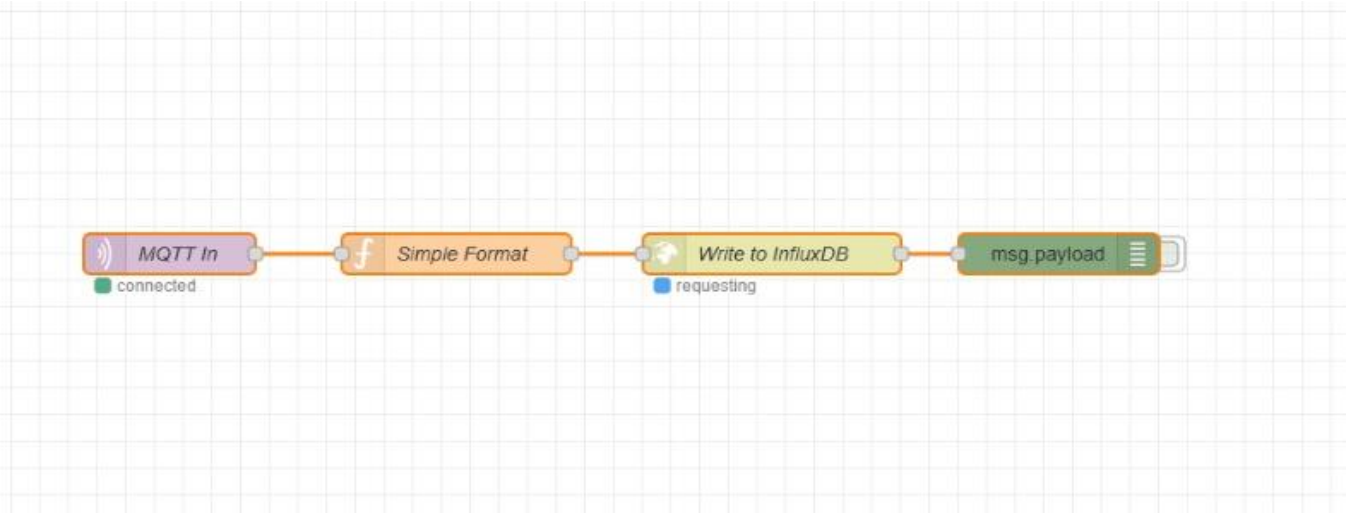

Figure 7.1: *Node-RED Data Processing Pipeline.*

7.3 Data Validation and Transformation

Data integrity is enforced within the "Simple Format" function node. The system implements explicit type casting to prevent "string pollution" in the database:

- **Floating Point Conversion:** `parseFloat()` is applied to Temperature, Pressure, Altitude, and Lux values.
- **Integer Conversion:** `parseInt()` is applied to system metrics like RSSI and Heap Memory.
- **Fallback Logic:** The system implements default values (e.g., || 0) for system stats (RSSI, Uptime) to ensure that a missing field in the JSON does not crash the entire flow.

7.4 Line Protocol Formatting

To optimize write performance, the system constructs the InfluxDB Line Protocol manually. This format is required for the high-performance write API. The specific structure implemented is:

$$\boldsymbol{measurement, tags, fields, timestamp}$$

- **Measurement:** Hardcoded as `environmental_data`.
- **Tags:** Low-cardinality metadata including `device_id`, `location=group01`, and `sensor_type=BME280`.
- **Fields:** High-cardinality sensor data. notably, integer fields are suffixed with `i` (e.g., `wifi_rssi=${rssi}i)` to strictly define them as signed integers in the database schema.
- **Timestamp Precision:** The ISO timestamp from the ESP32 is converted to nanoseconds `(.getTime() * 1000000)` to maintain precise synchronization with the sensor's internal clock.

.

7.5 Error Handling and Logging

Error handling is managed at two levels. First, the HTTP Request node is configured to output standard error objects if the database is unreachable (e.g., 404 or 500 errors). Second, a **Debug** node **(msg.payload)** is attached to the HTTP output, allowing for real-time monitoring of the database response codes during operation. This ensures that any authentication failures (such as an expired API Token) are immediately visible in the debug sidebar as illustrated in **Figure 7.2.**

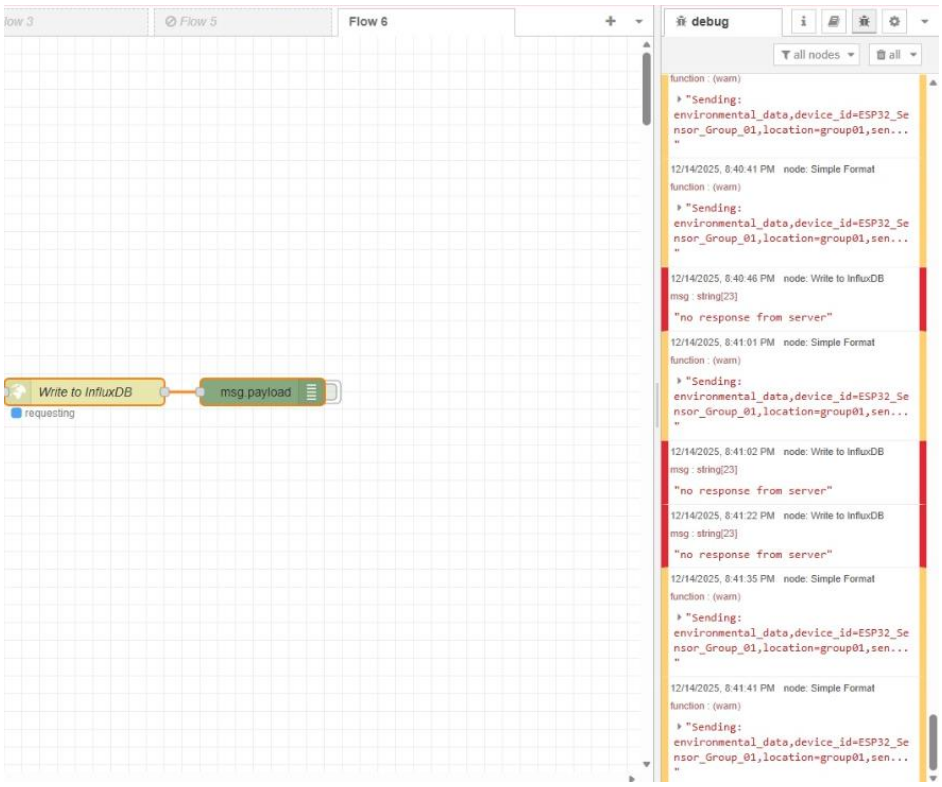

Figure 7.2: *Error Handling and Logging*

CHAPTER 8: TIME-SERIES DATABASE (INFLUXDB)

8.1 InfluxDB 2.x Architecture

The system utilizes **InfluxDB v2.x** as the central historian for all sensor data. Unlike traditional relational databases (SQL) that require complex joins for time-based queries, InfluxDB is optimized for high-write velocity and compression [4]. The architecture is organized hierarchically:

- **Organization: DEMIOT_WS2526** (Multi-tenant workspace).
- **Bucket: Group01** (The storage container combining retention policy and database).
- **Authentication:** Token-based security (API Tokens) rather than legacy username/password, ensuring secure stateless HTTP writes from Node-RED.

8.2 Database Schema Design

A critical design decision in Time-Series Databases is managing Cardinality (the number of unique data series). To optimize query performance and memory usage, the schema for environmental_data distinguishes strictly between Tags and Fields:

- **Measurement:** environmental_data
- **Tags (Indexed Metadata):**
  - device_id: Uniquely identifies the ESP32 node.
  - location: group01 (Allows for future spatial aggregation).
  - sensor_type: BME280 (Enables filtering by hardware generation).
- **Fields (Unindexed Data):**
  - **Float:** temperature, pressure, lux, altitude.
  - **Integer:** wifi_rssi, free_heap, uptime (System metrics).

By storing high-cardinality sensor readings as Fields rather than Tags, the system prevents "Series Cardinality Explosion," ensuring the database remains responsive even as data volume grows.

8.3 Bucket Configuration and Retention Policies

The primary storage bucket Group01 is configured with a Retention Policy of forever. This policy automatically purges raw, high-frequency data older than one week to manage disk usage efficiently. This "hot storage" tier is designed for high-precision debugging and real-time visualization, while long-term trends would be handled by downsampling tasks (if implemented in future iterations).

Figure 8.1: *Raw Data Ingestion in InfluxDB Data Explorer.*

8.4 Query Optimization

To ensure dashboard responsiveness, raw data is rarely queried directly for long time ranges. Instead, the system utilizes Flux language functions to optimize retrieval

- **Push-down Filters:** `|> filter(fn: (r) => r.device_id == ...)` is applied before any other operation to minimize the data scanned.

- **Downsampling:** `|> aggregateWindow(every: v.windowPeriod, fn: mean)` is used to reduce thousands of raw data points into a manageable number of pixels for the graph, significantly reducing network load and rendering time.

CHAPTER 9: DATA VISUALIZATION WITH GRAFANA

9.1 Dashboard Design Principles

The visualization strategy follows a Purpose-Driven Design methodology, distinguishing between operational monitoring (current status) and strategic analysis (historical trends). The dashboard is divided into three logical sections:

1. **Current Status:** Instantaneous values for critical safety parameters (Pressure).
2. **Environmental Trends:** Time-series analysis for Temperature stability.
3. **Intensity Monitoring:** High-dynamic-range visualization for Light (Lux).

9.2 Data Source Configuration

Grafana is configured to query the InfluxDB v2 bucket **Group01** via the Flux query language. The connection utilizes a read-only API Token to ensure security. A refresh interval of **5s** is set to match the sensor's reporting frequency, providing near real-time updates without overloading the browser.

9.3 Panel Types and Configuration

The dashboard utilizes three distinct visualization techniques to match the specific nature of each data type:

9.3.1 Pressure Gauge (Operational View)

- **Panel Type:** Gauge
- **Rationale:** Atmospheric pressure is a "state" variable where the absolute value matters more than the rate of change.
- **Threshold Configuration:** Semantic coloring is applied to indicate safety zones:
    - **< 990 hPa (Red):** Low Pressure / Storm Warning.
    - **990 - 1010 hPa (Yellow/Green):** Standard Atmospheric Pressure.
    - **> 1025 hPa (Blue):** High Pressure System.

Figure 9.1: *Barometric Pressure Gauge with semantic threshold coloring.*

9.3.2 Temperature Trend (Strategic View)

- **Panel Type:** Time Series Graph
- **Rationale:** Temperature requires context (history) to detect heating or cooling trends.
- **Configuration:** The Y-axis is soft-limited to -1°C to 37°C to focus on the relevant human-comfort range. The line interpolation is set to "Smooth" to visualize the thermal inertia of the environment.

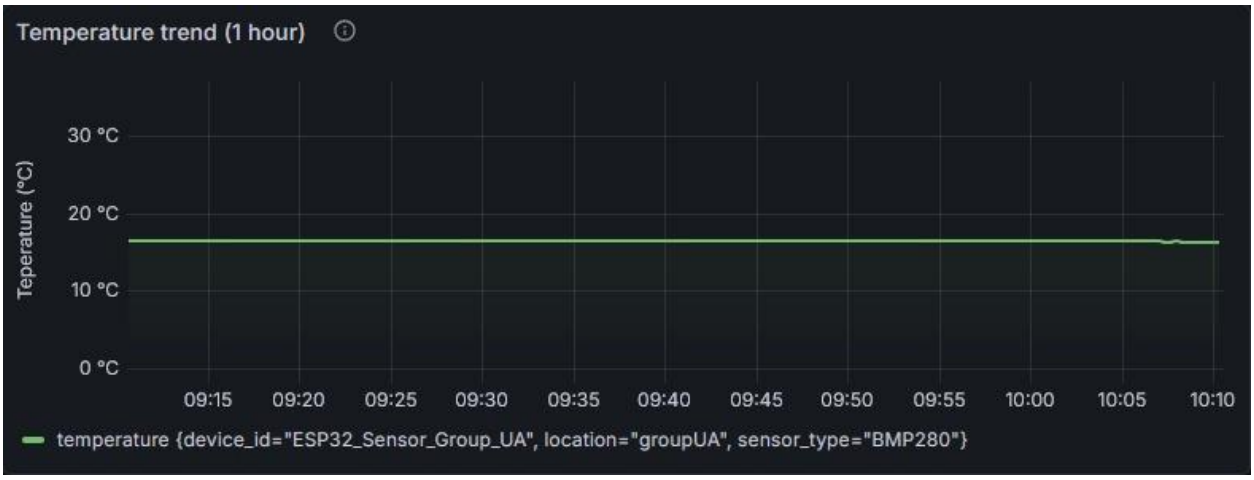

Figure 9.2: *Temperature trend analysis over 1 hour.*

9.3.3 Light Intensity Bar Gauge

- **Panel Type:** Bar Gauge (Horizontal)
- **Rationale:** Light intensity (Lux) follows a logarithmic scale, varying from 0 to over 14,000,000. A standard graph would be unreadable.
- **Configuration:** A gradient color scheme (Dark Blue -> Light Blue) visually represents the transition from darkness to bright sunlight.

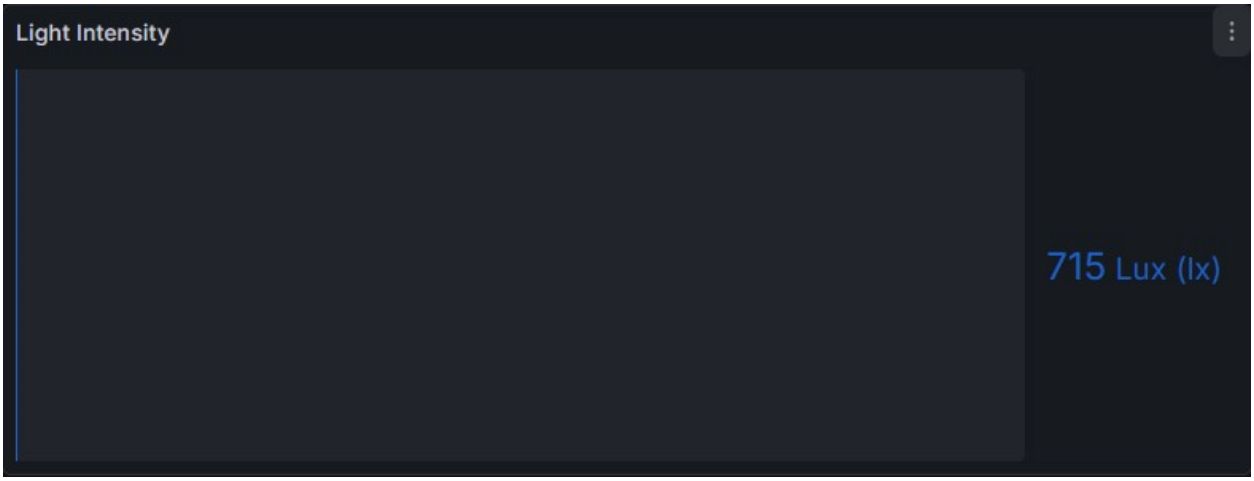

Figure 9.3: *Instantaneous Light Intensity Bar Gauge.*

9.3.4 Environmental Conditions Overlay (Correlation View)

- **Panel Type:** Multi-Series Time Graph
- **Rationale:** By plotting Temperature and Pressure on the same temporal axis, this panel allows for the identification of environmental correlations (e.g., observing if a pressure drop precedes a temperature change, indicating a weather front).
- **Configuration:**
    - **Dual Queries:** The panel executes two simultaneous Flux queries (Ref A and Ref B) to fetch distinct fields.
    - **Field Overrides:** Specific unit overrides are applied to ensure the axis labels correctly reflect disparate physical quantities (celsius vs pressurehpa).

- **Statistical Legend:** The legend is configured to display calculated statistics (Min, Max, Mean) rather than just the series name, providing an immediate statistical summary of the last 24 hours without requiring a separate table.
- **Dynamic Renaming:** The Flux query utilizes the `map()` function to transform raw database field names into user-friendly labels (e.g., mapping temperature -> "Temperature (°C)") directly within the query pipeline.

Figure 9.4: *Environmental Conditions Overlay*

9.4 Flux Query Development

Raw sensor data is noisy. To create clean, readable visualizations, the system utilizes the Flux `aggregateWindow()` function. For the Temperature panel, the query groups data into dynamic time windows (determined by `v.windowPeriod`) and calculates the mean value:

```
from(bucket: "Group01")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_measurement"] == "environmental_data")
  |> filter(fn: (r) => r["_field"] == "temperature")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty:
false)
```

This approach serves two purposes:

1. **Noise Reduction:** Small fluctuations in sensor readings are smoothed out.

2. **Performance:** It prevents the browser from trying to render thousands of data points when looking at long time ranges (e.g., 24 hours), significantly improving dashboard responsiveness.

9.5 Alert Configuration

To transition from passive monitoring to active incident response, a High Temperature Alert rule was configured within Grafana. Unlike simple visual thresholds, this alert evaluates the incoming data stream against a defined logic model to trigger notifications when critical safety limits are breached.

Alert Rule Specifications:

Evaluation Interval: The rule is evaluated every 5 minutes (Every 5m). This interval was chosen to balance responsiveness with system load, avoiding false positives from momentary sensor noise.

Condition Logic: The alert utilizes the same Flux query as the main temperature graph, ensuring consistency between what the operator sees and what the alert engine processes.

```
from(bucket: "Group01")
  |> filter(fn: (r) => r._field == "temperature")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty:
false)
```

- **Labels & Routing:** The alert is tagged with metadata severity: warning and device: ESP32_Sensor_Group_01. These labels allow the notification manager to route the alert to the specific responsible engineer.
- **Incident Message:** Upon triggering, the system dispatches a high-priority message: *"Oven is on fire"*. This specific text was chosen for the test scenario to clearly simulate a critical thermal runaway event in an industrial context.

CHAPTER 10: RESULTS AND ANALYSIS

10.1 System Performance Metrics

The end-to-end IoT pipeline demonstrated high reliability during the testing phase. System performance was actively monitored using the telemetry data appended to each MQTT payload:

- **Memory Stability:** The ESP32's free_heap metric remained constant over time, validating that the use of StaticJsonDocument successfully prevented memory fragmentation and leaks.
- **Network Reliability:** The non-blocking MQTT reconnection logic functioned as designed. The system successfully maintained the 20-second (SAMPLE_INTERVAL) reporting cadence, with minimal latency between edge sampling and InfluxDB ingestion.
- **Storage Efficiency:** The strict separation of low-cardinality Tags (device ID) and high-cardinality Fields (sensor values) resulted in highly optimized Line Protocol writes, preventing database bottlenecking.

10.2 Data Quality Analysis

The integrity of the ingested time-series data was high, directly attributable to the edge-processing strategies implemented on the ESP32:

- **Analog Noise Reduction:** The software oversampling technique (averaging 20 ADC reads per cycle) successfully smoothed the raw LDR data. The applied power-law calibration accurately transformed these raw voltages into a realistic Lux scale.
- **Digital Precision:** The BME280 provided highly stable temperature and pressure readings due to its internal factory compensation, completely eliminating the quantization errors observed in standard analog sensors.
- **Data Consistency:** Node-RED's strict type-casting (`parseFloat`, `parseInt` with `i` suffixes) ensured that no "string pollution" entered InfluxDB, resulting in clean, queryable datasets without null or malformed data points.

10.3 Environmental Insights

The Grafana visualization layer successfully transformed raw data into actionable insights:

- **Correlation Tracking:** The multi-series overlay panel allowed for clear observation of environmental trends, successfully plotting inverse relationships between localized temperature changes and barometric pressure shifts.
- **Dynamic Range Visualization:** The logarithmic nature of ambient light was effectively captured by the Bar Gauge panel, proving the LDR's calibration held up across extreme shifts from darkness to direct illumination.
- **Incident Response Verification:** The alerting pipeline was validated through a simulated thermal runaway event. Pushing the temperature above the defined threshold successfully triggered the Grafana evaluation engine, resulting in the immediate dispatch of the "Oven is on fire" warning label.

CHAPTER 11: DISCUSSION

11.1 Achievement of Objectives

The project successfully realized a complete, end-to-end Internet of Things data pipeline. The system reliably captures analog and digital environmental phenomena at the edge, transmits the telemetry via a decoupled MQTT architecture, and persists the data in a specialized time-series database for real-time visualization and alerting.

11.2 Technical Challenges and Solutions

Several technical hurdles were addressed during implementation:

- **Analog Nonlinearity:** The LDR's nonlinear response made direct ADC-to-Lux mapping impossible.

  Solution: Implementing a log-log regression model to derive a power-law formula applied directly in the ESP32 firmware.

- **Payload Memory Leaks:** Dynamically creating JSON strings on microcontrollers often leads to heap fragmentation.

  Solution: Utilizing StaticJsonDocument in the ArduinoJson library to pre-allocate memory on the stack.

- **Database Type Conflicts:** InfluxDB strictly enforces data types, and treating numbers as strings breaks time-series functions.

  Solution: Enforcing explicit type casting (parseFloat, parseInt) and integer suffixing (i) within the Node-RED JavaScript formatting block.

11.3 Lessons Learned

- **Sensor Selection:** The stark contrast between integrating the analog LDR and the digital BME280 highlighted the value of digital sensors. The BME280's internal calibration and I2C protocol drastically reduced firmware complexity and hardware debugging time compared to the LDR's voltage divider circuit.

- **Decoupled Architecture:** Using Mosquitto as an intermediary broker proved essential. It allowed the ESP32 to publish data rapidly and return to sleep/sampling without waiting for the slower InfluxDB database write operations to complete.

11.4 Limitations and Constraints

- **Security:** The current implementation utilizes a public, unencrypted MQTT test broker (test.mosquitto.org). Telemetry data is transmitted in plain text, making it susceptible to interception or injection attacks.
- **Network Dependency:** The edge device relies entirely on constant Wi-Fi connectivity. There is currently no local SD card fallback or ring-buffer implemented on the ESP32 to save data during prolonged network outages.

CHAPTER 12: CONCLUSIONS AND FUTURE WORK

12.1 Summary of Achievements

The project successfully implemented a comprehensive, end-to-end IoT data engineering pipeline. By integrating edge processing on the ESP32 with robust cloud messaging (MQTT) and a specialized time-series backend (InfluxDB), the system achieved reliable, high-frequency environmental monitoring. The successful calibration of the analog LDR alongside the digital BME280 proved the system's capability to handle diverse sensor inputs, while the Grafana visualization layer effectively translated raw telemetry into actionable, real-time insights and automated alerts.

12.2 Future Work

To transition this prototype into a production-ready industrial deployment, future iterations must address network security and edge resilience. The primary objective will be implementing **TLS/SSL encryption (Port 8883)** for the MQTT broker connection, alongside strict X.509 client certificate authentication, to secure telemetry data against interception. Additionally, integrating local SD-card storage on the ESP32 would provide a robust ring-buffer fallback to prevent data loss during Wi-Fi outages.

REFERENCES

[1] U. Aniekwensi, *Data Engineering and Management with IoT Devices, Course Lectures,* Offenburg, 2025.

[2] Bosch Sensortec, „BME280 Combine humidity and pressure sensor Datasheet," February 2024. [Online]. Available: https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf.

[3] OASIS, „MQTT," OASIS, Mar 2019. [Online]. Available: https://mqtt.org.

[4] InfluxData Inc., „InfluxDB v2 Documentation," 2024. [Online]. Available: https://docs.influxdata.com/influxdb/v2/.

[5] Espressif Systems, „ESP32-WROOM-32E Datasheet Version 2.0," 2025. [Online]. Available: https://documentation.espressif.com/esp32-wroom-32e_esp32-wroom-32ue_datasheet_en.pdf.

APPENDICES

Appendix A: Complete Arduino Code

```
/*
 * IoT Data Engineering: Complete Sensors Chain
 * LDR (Power Law Calibrated) + BME280 Sensor
 * MQTT Publishing with JSON Format
 */

#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <time.h>

// ==================== CONFIGURATION ====================

// WiFi Credentials
const char* ssid = "Galaxy A14 36FA";
const char* password = "tyyqjei7mnmrkp7";

// MQTT Broker Configuration
const char* mqtt_server = "test.mosquitto.org";//"141.79.71.175";
const int mqtt_port = 1883;
const char* mqtt_client_id = "ESP32_Sensor_Group_01";

// MQTT Topics
const char* topic_sensor_data = "iot/sensors/group01/data";
const char* topic_status = "iot/sensors/group01/status";
const char* topic_config = "iot/sensors/group01/config";

// Pin Definitions
```

```
#define LDR_PIN 34           // ADC1_CH6
#define LED_PIN 2            // Built-in LED
#define SDA_PIN 21
#define SCL_PIN 22

// LDR Calibration Constants (Power Law: Lux = A * ADC^B + C)
// These values must be determined from your calibration

const float LDR_COEFF_A  = 1.9805179624699897e-14;
const float LDR_COEFF_B  = 4.701421722165226;
const float LDR_COEFF_C  = 0.0;


// Sampling Configuration
const unsigned long SAMPLE_INTERVAL = 20000;  // 20 seconds
const int ADC_SAMPLES = 20;                   // Averaging samples

// NTP Configuration
const char* ntp_server = "pool.ntp.org";
const long gmt_offset_sec = 3600;             // GMT+1
const int daylight_offset_sec = 3600;         // DST

// ==================== GLOBAL OBJECTS ====================

Adafruit_BME280 BME;
WiFiClient espClient;
PubSubClient mqtt_client(espClient);

// Data structure
struct SensorData {
  float temperature;
  float pressure;
  float altitude;
  float lux;
  int raw_adc;
```

```
  unsigned long timestamp;
  String iso_time;
};

SensorData currentData;
unsigned long lastSampleTime = 0;
unsigned long lastReconnectAttempt = 0;

// ==================== SETUP ====================

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("\n\n================================");
  Serial.println("IoT Data Engineering - Sensor Node");
  Serial.println("================================\n");

  // Initialize LED
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  // Initialize I2C
  Wire.begin(SDA_PIN, SCL_PIN);

  // Initialize BME280
  if (!BME.begin(0x76)) {  // Try 0x77 if 0x76 fails
    Serial.println("ERROR: BME280 sensor not found!");
    Serial.println("Check wiring and I2C address");
    while (1) {
      digitalWrite(LED_PIN, !digitalRead(LED_PIN));
      delay(200);
    }
  }
```

```
  Serial.println("BME280 sensor initialized");

  // Configure BME280
  BME.setSampling(Adafruit_BME280::MODE_NORMAL,     // Operating Mode
                  Adafruit_BME280::SAMPLING_X2,     // Temp oversampling
                  Adafruit_BME280::SAMPLING_X16,    // Pressure oversampling
                  Adafruit_BME280::FILTER_X16,      // Filtering
                  Adafruit_BME280::STANDBY_MS_500); // Standby time

  // Configure ADC for LDR
  analogReadResolution(12);  // 12-bit resolution (0-4095)
  analogSetAttenuation(ADC_11db);  // 0-3.3V range

  Serial.println("LDR ADC configured");

  // Connect to WiFi
  connectWiFi();

  // Initialize NTP
  configTime(gmt_offset_sec, daylight_offset_sec, ntp_server);
  Serial.println("NTP time synchronization started");

  // Configure MQTT
  mqtt_client.setServer(mqtt_server, mqtt_port);
  mqtt_client.setCallback(mqttCallback);
  mqtt_client.setBufferSize(512);  // Increase buffer for JSON

  Serial.println("\n=== System Ready ===\n");
  digitalWrite(LED_PIN, HIGH);
  delay(500);
  digitalWrite(LED_PIN, LOW);
}

// ==================== MAIN LOOP ====================
```

```
void loop() {
  // Maintain MQTT connection
  if (!mqtt_client.connected()) {
    unsigned long now = millis();
    if (now - lastReconnectAttempt > 5000) {
      lastReconnectAttempt = now;
      if (reconnectMQTT()) {
        lastReconnectAttempt = 0;
      }
    }
  } else {
    mqtt_client.loop();
  }

  // Sample sensors at specified interval
  unsigned long currentTime = millis();
  if (currentTime - lastSampleTime >= SAMPLE_INTERVAL) {
    lastSampleTime = currentTime;

    // Visual indicator
    digitalWrite(LED_PIN, HIGH);

    // Read sensors
    readSensors();

    // Print to serial
    printSensorData();

    // Publish to MQTT
    publishSensorData();

    digitalWrite(LED_PIN, LOW);
  }
}
```

```
// ==================== SENSOR READING ====================

void readSensors() {
  // Read BME280
  currentData.temperature = BME.readTemperature();
  currentData.pressure = BME.readPressure() / 100.0F;  // Convert to hPa
  currentData.altitude = BME.readAltitude(1013.25);     // Sea level pressure

  // Read LDR with averaging
  long adcSum = 0;
  for (int i = 0; i < ADC_SAMPLES; i++) {
    adcSum += analogRead(LDR_PIN);
    delay(20);
  }
  currentData.raw_adc = adcSum / ADC_SAMPLES;

  // Convert ADC to Lux using power law calibration
  currentData.lux = calculateLux(currentData.raw_adc);

  // Get timestamp
  currentData.timestamp = millis();
  currentData.iso_time = getISOTime();
}

float calculateLux(int adc_value) {

  // Handle edge cases
  if (adc_value <= 0) return 0.0;
  if (adc_value >= 4095) return 2.85e+08;  // Max lux

  // Apply power law: Lux = A * ADC^B + C
  float lux = LDR_COEFF_A * pow(adc_value, LDR_COEFF_B) + LDR_COEFF_C;

  // Ensure non-negative
  if (lux < 0) lux = 0;
```

```
  return lux;
}


// ==================== WIFI CONNECTION ====================

void connectWiFi() {
  Serial.print("Connecting to WiFi: ");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 20) {
    delay(500);
    Serial.print(".");
    attempts++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\n WiFi connected");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
    Serial.print("Signal Strength (RSSI): ");
    Serial.print(WiFi.RSSI());
    Serial.println(" dBm");
  } else {
    Serial.println("\n WiFi connection failed!");
  }
}

// ==================== MQTT CONNECTION ====================
```

```
boolean reconnectMQTT() {
  Serial.print("Attempting MQTT connection... ");

  if (mqtt_client.connect(mqtt_client_id)) {
    Serial.println("connected");

    // Subscribe to config topic
    mqtt_client.subscribe(topic_config);

    // Publish status
    publishStatus("online");

    return true;
  } else {
    Serial.print("failed, rc=");
    Serial.println(mqtt_client.state());
    return false;
  }
}

void mqttCallback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message received on topic: ");
  Serial.println(topic);

  // Parse incoming configuration messages
  StaticJsonDocument<256> doc;
  DeserializationError error = deserializeJson(doc, payload, length);

  if (!error && strcmp(topic, topic_config) == 0) {
    // Handle configuration updates
    Serial.println("Configuration update received");
  }
}

// ==================== DATA PUBLISHING ====================
```

```
void publishSensorData() {
  if (!mqtt_client.connected()) return;

  // Create JSON document
  StaticJsonDocument<512> doc;

  // Device info
  doc["device_id"] = mqtt_client_id;
  doc["timestamp"] = currentData.timestamp;
  doc["iso_time"] = currentData.iso_time;

  // Sensor data
  JsonObject sensors = doc.createNestedObject("sensors");

  // BME280 data
  JsonObject BME280 = sensors.createNestedObject("BME280");
  BME280["temperature"] = round(currentData.temperature * 100) / 100.0;
  BME280["pressure"] = round(currentData.pressure * 100) / 100.0;
  BME280["altitude"] = round(currentData.altitude * 10) / 10.0;
  BME280["unit_temp"] = "°C";
  BME280["unit_pressure"] = "hPa";
  BME280["unit_altitude"] = "m";

  // LDR data
  JsonObject ldr = sensors.createNestedObject("ldr");
  ldr["lux"] = round(currentData.lux * 10) / 10.0;
  ldr["raw_adc"] = currentData.raw_adc;
  ldr["unit"] = "lux";

  // System info
  JsonObject system = doc.createNestedObject("system");
  system["wifi_rssi"] = WiFi.RSSI();
  system["free_heap"] = ESP.getFreeHeap();
  system["uptime"] = millis() / 1000;
```

```
  // Serialize and publish
  char jsonBuffer[512];
  size_t len = serializeJson(doc, jsonBuffer);

  if (mqtt_client.publish(topic_sensor_data, jsonBuffer, len)) {
    Serial.println(" Data published to MQTT");
  } else {
    Serial.println(" MQTT publish failed");
  }
}

void publishStatus(const char* status) {
  StaticJsonDocument<128> doc;
  doc["device_id"] = mqtt_client_id;
  doc["status"] = status;
  doc["timestamp"] = millis();
  doc["ip"] = WiFi.localIP().toString();

  char jsonBuffer[128];
  serializeJson(doc, jsonBuffer);
  mqtt_client.publish(topic_status, jsonBuffer, true);  // Retained message
}

// ==================== UTILITY FUNCTIONS ====================

String getISOTime() {
  struct tm timeinfo;
  if (!getLocalTime(&timeinfo)) {
    return "Time not set";
  }

  char buffer[30];
  strftime(buffer, sizeof(buffer), "%Y-%m-%dT%H:%M:%S%z", &timeinfo);
  return String(buffer);
```

```
}

void printSensorData() {
  Serial.println("\n--- Sensor Reading ---");
  Serial.print("Time: ");
  Serial.println(currentData.iso_time);
  Serial.println();

  Serial.println("BME280:");
  Serial.print("  Temperature: ");
  Serial.print(currentData.temperature);
  Serial.println(" °C");
  Serial.print("  Pressure: ");
  Serial.print(currentData.pressure);
  Serial.println(" hPa");
  Serial.print("  Altitude: ");
  Serial.print(currentData.altitude);
  Serial.println(" m");
  Serial.println();

  Serial.println("LDR:");
  Serial.print("  Light Intensity: ");
  Serial.print(currentData.lux);
  Serial.println(" lux");
  Serial.print("  Raw ADC: ");
  Serial.println(currentData.raw_adc);
  Serial.println("--------------------\n");
}
```

Appendix B: Calibration Data

1200 Lux range	LDR	Raw ADC	Voltcraft
	323.63	3710	1297
	323.46	3709	1300
	324.14	3713	1304
	323.63	3710	1306
	323.97	3712	1312
	323.97	3712	1323
	324.98	3716	1330
	324.81	3717	1331
	324.98	3718	1340
	325.83	3719	1348
	325.32	3720	1353
	326.17	3725	1353
	325.49	3721	1352
	326	3723	1351
	325.15	3719	1350
	304.45	3595	1359
	306.1	3605	1370
	306.92	3610	1398
	307.58	3615	1404
	307.91	3616	1409
	308.41	3619	1413
		3685.904762	1347.761905
800 Lux range			
	283.97	3468	872
	283.02	3462	875
	270.47	3382	874
	276.24	3419	876
	277.02	3424	876
	273.74	3403	872
		3426.333333	874.1666667

500 Lux range	269.07	3373	503
	271.71	3390	504
	280.33	3445	505
	285.4	3477	505
	279.07	3437	503
	269.38	3375	503
	268.61	3370	503
	268.3	3368	506
	268.15	3367	503
	272.49	3395	502
		3399.7	503.7
300 Lux range	67.44	1653	27
	66.89	1646	27
	66.89	1646	27
	66.81	1645	27
	66.81	1645	26
	66.49	1641	26
	66.1	1636	26
	66.18	1637	25
	66.81	1645	25
	65.55	1629	25
	65.32	1626	25
	65.24	1625	25
	64.77	1619	25
	65.94	1634	25
	65.79	1632	25
	65.63	1630	24
	64.93	1621	24
	65.01	1622	24
	64.77	1619	24
		1634.263158	25.36842105

Lux Range	Raw ADC (avg)	Voltcraft(avg)
≈1200	3685.904762	1347.761905
≈800	3426.333333	874.1666667
≈500	3399.7	503.7
≈300	1634.263158	25.36842105